Guía de empresa: del servidor del aula a la clase

Esta guía arma el cluster del aula de punta a punta. Primero se prepara la PC del profe como servidor, después se suman las PCs de los alumnos por Veyon y por último se da la clase: cargar datos, correr trabajos, mirar el tráfico, apagar una PC a propósito y cerrar sin perder nada. Al final hay dos variantes: el **aula simulada** en una sola compu y el armado **a mano** con `docker run`.

Para saber qué es cada pieza, mirá el README. Para ver cómo se conectan, abrí el mapa interactivo.

Los comandos llevan, antes, un comentario invisible (`<!-- prueba: ... -->`) que usa `pruebas/tutoriales.py`. Los de esta guía no se corren solos, porque necesitan Windows, Veyon o varias PCs. Se prueban en el workflow `empresa.yml` con `pruebas/aula.sh`.

---

1. Lo que necesitás

En la PC del profe (el maestro):

- Windows con **Docker Desktop** instalado y andando.
- Memoria: el maestro corre NameNode, ResourceManager, Hive, Spark History y la interfaz. Dale a Docker todo lo que puedas; 8 GB es un buen piso.
- Disco para HDFS. El maestro guarda **una copia de todo** lo que se cargue en la clase.
- Estar en la misma red que las PCs del aula.

En cada PC de alumno:

- Windows con **Docker Desktop** instalado (no hace falta abrirlo: el `.cmd` lo abre).
- No hace falta administrador ni abrir puertos.

En el aula:

- **Veyon** (o la herramienta de control del laboratorio) para mandar archivos y ejecutar un programa en varias PCs a la vez.
- Que las PCs lleguen a la del profe por **UDP 51820** y **TCP 8099**.

---

2. Bajar el paquete del aula

Del release `aula-v2` de GitHub (*Releases → Aula v2*) bajá a una carpeta de la PC del profe, por ejemplo `C:\aula`:

Archivo	Para qué
`bigdata-sandbox-empresa.tar.gz`	Imagen del maestro (amd64)
`bigdata-sandbox-trabajador.tar.gz`	Imagen de las PCs (amd64)
`bigdata-sandbox-trabajador-arm64.tar.gz`	Imagen de las PCs con Windows en ARM (sólo si hay)
`empresa-profe.cmd`	Se corre en la PC del profe
`empresa-alumno.cmd`	Se manda a las PCs

> **Estado al 10/11/2026:** el release `aula-v2` todavía no está publicado. Hasta que salga, el último es `aula-v1`, un prerelease del 29/10 con los mismos cinco archivos pero sin la vista Aula en vivo. Con los `.tar.gz` al lado, el aula anda sin internet. Sin ellos, los `.cmd` bajan las imágenes de Docker Hub (`:empresa` y `:empresa-trabajador`).

Si en el laboratorio hay una carpeta compartida de sólo lectura, podés dejar ahí el `.tar.gz` del trabajador. El alumno lo busca en la carpeta que le pases como tercer argumento.

---

3. Armar el servidor: arrancar el maestro

En la PC del profe, hacé doble clic en `empresa-profe.cmd`, o desde una consola:

```
C:\aula\empresa-profe.cmd
```

Qué hace, en orden:

1. Muestra el hardware de la PC (nombre, IP, RAM y CPU) y abre Docker Desktop si estaba cerrado.
2. Carga la imagen desde `bigdata-sandbox-empresa.tar.gz` y te dice cuánto tardó. Si el archivo no está, la baja de Docker Hub.
3. Te avisa si faltan `bigdata-sandbox-trabajador.tar.gz` o `empresa-alumno.cmd` en la carpeta: Veyon los manda desde ahí.
4. Crea el contenedor `bigdata-maestro`:
   - con el volumen `bigdata_aula` y los datos en `%USERPROFILE%\bigdata-aula`;
   - publica el UDP 51820 y el TCP 8099 hacia la red;
   - deja las webs sólo en `127.0.0.1`;
   - genera el **código de la clase** (4 letras).
5. Si al lado hay un `plano-aula.json` de otra clase, lo copia al maestro: sus filas son los racks.
6. Muestra el cartel con todo lo que vas a necesitar:

```
======================================================================
  BigData Sandbox - el cluster del aula
    IP de esta PC en el aula: 192.168.1.50
    Codigo de la clase:       K7MX
  En Veyon, con las PCs seleccionadas:
    1. Transferencia de archivos: bigdata-sandbox-trabajador.tar.gz y empresa-alumno.cmd
       (desde esta carpeta: C:\aula)
    2. Ejecutar programa:
       cmd /c ""C:\Users\...\empresa-alumno.cmd" 192.168.1.50 K7MX"
  Interfaz (vista Aula): http://localhost:8080/#aula
======================================================================
```

7. Espera a que Hadoop diga «Cluster LISTO» (con un tope de 10 minutos), te dice cuánto tardó y abre **http://localhost:8080/#aula**.

Firewall: la primera vez, Windows pregunta si deja entrar conexiones para Docker. Aceptá en redes **privadas**: es lo que abre el UDP 51820 y el TCP 8099.

Si la IP no es la del aula: si la PC tiene varias placas de red (Wi-Fi, cable, VPN), el cartel lista todas debajo. Usá la de la red del laboratorio en la línea de Veyon.

La línea queda también en `%USERPROFILE%\bigdata-aula\veyon.txt`, para copiar y pegar.

---

4. Conectar las PCs del aula (Veyon)

En Veyon, seleccioná las PCs de los alumnos y hacé:

1. **Transferencia de archivos:** mandá `bigdata-sandbox-trabajador.tar.gz` y `empresa-alumno.cmd` desde la carpeta del profe.
2. **Ejecutar programa:** pegá la línea del cartel (la misma que aparece en la vista Aula, en *Sumar PCs*, con el botón *Copiar la línea para Veyon*).

En cada PC, `empresa-alumno.cmd`:

1. Abre Docker Desktop si estaba cerrado, y espera que esté listo.
2. Carga la imagen del trabajador: la busca al lado del `.cmd`, en la carpeta que le pases o en la carpeta del usuario, y si no la encuentra la baja de Docker Hub. En una PC con Windows en ARM usa la de arm64.
3. Le pide a la PC del profe un **nombre** (`pc01`, `pc02`...). Si el profe todavía no arrancó, espera.
4. Crea el contenedor `bigdata-trabajador` con ese nombre. El túnel se levanta solo y se suman el DataNode y el NodeManager.
5. Espera hasta 12 minutos a que la PC quede adentro del cluster y lo muestra.

Si preferís hacerlo PC por PC, a mano en una consola de la PC del alumno:

```
empresa-alumno.cmd 192.168.1.50 K7MX
```

Con la imagen en una carpeta compartida:

```
empresa-alumno.cmd 192.168.1.50 K7MX \\SERVIDOR\Compartida\bigdata
```

---

5. La vista Aula

Abrí **http://localhost:8080/#aula** (tecla `a`) y proyectala con la *Demostración* de Veyon, en **Pantalla completa**.

- Arriba, los **números grandes**: PCs en el cluster, memoria para trabajos, cores y bloques en HDFS.
- El **frente del aula** es el maestro, con lo que corre: NameNode, ResourceManager, Hive, Spark History y el hub del túnel.
- **Cada fila es un rack.** Cada PC tiene su lámpara:
  - ámbar mientras se suma;
  - verde cuando está arriba;
  - ámbar si deja de contestar;
  - roja si se cayó.
- **Tocá una PC** para ver su detalle: IP del túnel, último latido, bloques, memoria y cores usados.
- **Lo último** muestra los eventos: PCs que se suman, se caen o vuelven.

Si una PC no aparece, mirá en esa PC:

```
empresa-alumno.cmd estado
```

---

6. El plano: que las filas sean las del aula

De fábrica, las PCs se agrupan de a 5 por fila (`pc01` – `pc05` en `/fila1` , `pc06` – `pc10` en `/fila2` ...). Si el aula no está así:

1. En la vista Aula, tocá **Editar el plano**.
2. Acomodá las PCs en sus filas y lugares. Hay hasta 8 filas de 12 lugares.
3. **Guardar en el maestro.** El plano también queda como borrador en el navegador.
4. **Exportá** `plano-aula.json` y dejalo al lado de `empresa-profe.cmd` : la próxima clase se carga solo.

Hadoop lee el rack de cada PC **cuando su DataNode se registra**. Conviene armar el plano antes de sumar las PCs; si no, las que ya estaban toman su fila nueva recién cuando se reinician.

---

7. Cargar el dataset

Desde la PC del profe, subí un archivo o una carpeta (de un pendrive o de una carpeta compartida) a HDFS:

```
empresa-profe.cmd cargar E:\datasets\ventas
```

Por defecto va a `/aula/datos/<nombre>` . Con un tercer argumento elegís otra ruta de HDFS:

```
empresa-profe.cmd cargar E:\datasets\ventas.csv /aula/datos/ventas.csv
```

El `.cmd` mide la velocidad (MB/s), lo anota en `%USERPROFILE%\bigdata-aula\cargas.txt` y al final muestra **cómo quedaron repartidos los bloques**: cuántos hay en cada PC y en qué rack. Con la réplica de fábrica del aula (2), cada bloque queda en el maestro y en una PC.

---

8. Correr trabajos

La interfaz del maestro es la misma de full: Tablas, Motores, Pipeline, ML, Trabajos. Mientras un trabajo corre, en la vista Aula aparecen **cuadraditos** adentro de cada PC, uno por contenedor de YARN. El que tiene borde es el ApplicationMaster.

Para lanzar algo desde la terminal del maestro, abrí el cajón de terminal de la interfaz (`empresa-profe.cmd` la prende sólo para localhost) o entrá con Docker:

```
docker exec -it bigdata-maestro bash
```

Adentro, por ejemplo, un conteo de líneas por sucursal con Hive:

```
beeline -u jdbc:hive2://localhost:10000 -n root -e "SELECT COUNT(*) FROM ventas;"
yarn node -list
```

`yarn node -list` muestra un NodeManager por PC sumada.

9. La red del aula

En la vista Aula, la sección **La red del aula** muestra en vivo el tráfico de cada PC, separado por tipo:

- **HDFS:** copias de bloques;
- **shuffle:** lo que un trabajo se pasa entre PCs;
- **YARN:** latidos y control;
- **otro.**

La tabla de abajo dice qué PCs mueven más. La línea de 50 Mbit/s es una referencia del dibujo, no una medición de tu aula.

Para que se note el shuffle, corré un join o un `GROUP BY` sobre un dataset grande: vas a ver crecer la parte de *shuffle* en las PCs que participan.

10. Apagar una PC a propósito

Es el experimento más lindo de la clase:

1. Lanzá un trabajo largo.
2. Con Veyon, **apagá una PC** que tenga cuadraditos (o desenchufala).
3. Mirá en **Si una PC se cae**:
    - **~10 s** sin latidos: la lámpara pasa a ámbar;
    - **40 s**: el NameNode la da por muerta y HDFS empieza a copiar sus bloques en otras PCs;
    - **60 s**: YARN la marca LOST y **reintenta sus tareas en otra PC**.
4. El trabajo termina igual. La línea de tiempo de la caída resume cuántos contenedores se perdieron, cuántos bloques se re-copiaron y cuánto tardó.

Si la PC vuelve a prenderse, para volver a sumarla corré otra vez la línea de Veyon en esa PC.

11. Terminar la clase: «Ya pueden apagar»

Antes de que los alumnos apaguen, mirá la tarjeta de arriba de la vista Aula:

- **«Todavía no apaguen las PCs»** te dice el porqué. Por ejemplo: «el maestro tiene 120 de 150 bloques: HDFS está copiando lo que falta». Esperá.
- **«Ya pueden apagar»**: el maestro tiene una copia de cada bloque y no falta ninguno. Se puede apagar el aula sin perder nada.

En cada PC, para salir prolijo:

```
empresa-alumno.cmd salir
```

Para apagar el maestro conservando los datos para la próxima clase:

```
empresa-profe.cmd apagar
```

La próxima vez, `empresa-profe.cmd` sin argumentos lo vuelve a prender con lo que tenía.

12. Empezar de cero (otra clase)

Para otra clase, borrá el maestro y su HDFS: genera un código y una clase nuevos. Pide escribir `SI`.

```
empresa-profe.cmd cero
```

Las PCs que se unan a la clase nueva borran solas lo que tenían de la anterior.

---

13. Variante: el aula simulada en una sola compu

Sirve para probar todo antes de ir al laboratorio. Levanta un maestro y dos PCs (`pc01`, `pc02`) como contenedores en la red de Docker (`RED=docker`), con 2 PCs por fila. Necesita unos 8 GB de RAM libres para Docker.

Desde la raíz del repo, o con `empresa/aula-simulada.yml` copiado a cualquier carpeta:

```
docker compose -f empresa/aula-simulada.yml up -d
docker compose -f empresa/aula-simulada.yml logs -f maestro
```

Esperá el cartel «Cluster LISTO» en los logs y abrí **http://localhost:8080/#aula**. Para entrar a una «PC» o ver el estado del maestro:

```
docker exec -it pc01 bash
docker exec maestro estado
```

Para sumar una tercera PC, copiá el bloque de `pc02` en el `.yml` y cambiá el número. Para apagar y borrar todo:

```
docker compose -f empresa/aula-simulada.yml down -v
```

> En la simulada, el tráfico no pasa por el túnel, así que la sección *La red del aula* tiene menos para mostrar. Para probar el túnel en una compu, `pruebas/aula.sh` tiene los escenarios `aula-wg` y `aula-wg-caida`.

---

14. Variante: a mano con `docker run` (Linux o Mac)

Si alguna PC no es Windows, se puede hacer lo mismo que los `.cmd` a mano.

El maestro se corre en la PC del profe. Cambiá `192.168.1.50` por su IP en el aula y `K7MX` por un código de 4 letras:

```
docker run -d --name bigdata-maestro --hostname maestro \
  --cap-add NET_ADMIN --device /dev/net/tun \
  --sysctl net.ipv4.ip_forward=1 --stop-timeout 20 \
  -p 51820:51820/udp -p 8099:8099 \
  -p 127.0.0.1:8080:8080 -p 127.0.0.1:9870:9870 -p 127.0.0.1:8088:8088 \
  -v bigdata_aula:/hadoop -v "$HOME/bigdata-aula:/datos" \
  -e ROL=maestro -e RED=wireguard -e CODIGO=K7MX -e IP_AULA=192.168.1.50 \
  franki678/bigdata-sandbox:empresa
```

Cada PC hace dos pasos.

Primero reserva su nombre. Esto imprime una línea `NODO=pcNN`:

```
docker run --rm --entrypoint /opt/sandbox/aula/red.sh \
  -e RED=wireguard -e ROL=trabajador -e MAESTRO=192.168.1.50 -e CODIGO=K7MX \
  franki678/bigdata-sandbox:empresa-trabajador reservar
```

Después arranca con **ese mismo nombre** como hostname. Es obligatorio, porque Tez y Spark se anuncian con el nombre de la máquina:

```
docker run -d --name bigdata-trabajador --hostname pc07 \
  --cap-add NET_ADMIN --device /dev/net/tun --stop-timeout 20 \
  -e RED=wireguard -e MAESTRO=192.168.1.50 -e CODIGO=K7MX \
  franki678/bigdata-sandbox:empresa-trabajador
```

Si la Mac no trae `/dev/net/tun`, sacá ese `--device`. El túnel usa el módulo del kernel si está; si no hay ninguno de los dos, no levanta.

---

15. Si algo falla

Síntoma	Qué mirar
Las PCs no se suman («no contesta»)	Que `empresa-profe.cmd` esté andando, que la IP de la línea sea la de la red del aula y que el firewall deje pasar el UDP 51820 y el TCP 8099 (red privada)
«el codigo '...' no es el de esta clase»	El código cambió (se hizo `cero`): copiá la línea nueva de la vista Aula
«este contenedor se tiene que llamar pcNN»	El contenedor se creó sin reservar el nombre: volvé a correr `empresa-alumno.cmd`
La PC queda en ámbar	Su túnel no llega. Mirá `empresa-alumno.cmd estado` en esa PC. Si la red corta paquetes grandes, el túnel baja solo a MTU 1280
No hay cuadraditos aunque corra un trabajo	No hay PCs con NodeManager vivo: el maestro no corre tareas (salvo `MAESTRO_NM=si`)
«Todavía no apaguen» no cambia	Faltan bloques porque se apagaron PCs antes de tiempo: volvé a sumarlas hasta que el maestro tenga todo
Estado del maestro	`empresa-profe.cmd estado`, o `docker logs bigdata-maestro`